

Agent 跑通了，怎么证明它能上线？

从 0 到 1 搭一套最小可用的 Agent Evaluation Framework —— 八步法全图解

"Agent 最危险的不是报错，而是做错了还显示'已完成'。"

来源：X @ClorisSignal 长文 (2026-09)

Dean 转发推荐

复现：agent-eval/exercise 零依赖 Python

疑问即目录 (12 问)

1. Demo 跑通了，为什么还要 Eval?
2. 第一步为什么不是选框架，而是写 Charter?
3. 什么叫"做完了"? Outcome 条件 + Hard Failure
4. 30 条数据集怎么配? 为什么切四份?
5. Agent 评测和聊天模型评测差在哪? Outcome vs Trajectory
6. 谁来打分? Rules + Judge + Human 三层分工
7. LLM Judge 自己会错，怎么校准?
8. 跑成功一次为什么不算数? pass@k vs pass^k
9. 80%→83% 为什么不能下结论? 置信区间与切片
10. 指标怎么变成 Release Gate? shadow / canary
11. 上线之后的闭环: 失败回流 + Component Lift
12. 今天就想动手，最小的一套怎么搭?

Q1 Demo 跑通了，为什么还要 Eval?

文章开头就是一个真实翻车现场：一个资料研究 Agent 交回 **结构完整、带引用、页面显示"已完成"** 的报告——随手点开三条引用链接：**一条失效、一条根本支撑不了结论、一条只是搜索摘要**。同一问题重跑一遍，结论又变了。

⚠️ Agent 最危险的失败模式：**静默失败 (silent failure)** ——没有报错、没有异常、界面还是绿色的"完成"。传统软件挂了会抛异常，Agent 挂了会**礼貌地交一份看起来很对的东西**。

所以 Eval (评测) 回答的从来不是"它能不能跑"，而是三个问题：**任务有没有做成？失败发生在哪里？新版本能不能发布？** 文章用一条 v1→v2 的发布决策把全流程串起来：v2 换了模型、改了 Prompt、加了搜索工具——**它能不能替换 v1 交给真实用户？** 整个评测框架就是为了拿到这个决定的证据。



Q2 第一步为什么不是选框架，而是写 Charter?

很多团队搭 Eval 的第一步是搜"Agent Eval 用什么框架"，然后开始比平台、比 Judge 模型、比指标。**工具很快装好了，真正要做的决定却始终没写下来**。同一套 Agent，因为要做的决策不同，评测可以完全不同：

要做的决策	评测重点
两个模型选一个	同批任务上的质量、成本、延迟
能否开放自动退款	越权操作、错误退款 = 硬门槛
只改了一段 Prompt	目标问题修好了吗 + 其他场景回归了吗

OpenAI 把这一阶段叫 **Specify**：先写清系统用途、关键决策、成功条件和要避免的行为。落到工程上就是一份 **eval-charter.yaml**，两个最容易被忽略的字段：

① **system_under_test** 要写完整——Agent 的结果来自模型+Prompt+检索+工具+工作流+权限+运行环境。只记"用了哪个模型"，几周后无法复现当时的结果。

② **unit_of_evaluation** 要先定——评一个 turn、一段对话，还是一次完整 task？资料研究 Agent 的价值在整项任务，所以选**一次完整 run**。

🔥 写完 Charter 时，**一次模型都还没跑**，但"为什么评、评谁、拿什么比、什么错误绝不能发生"这四件事已经钉死了。

Q3 什么叫"做完了"? Outcome 条件 + Hard Failure

Agent 最会制造一种错觉：**它做了很多步骤，所以任务应该完成了**。搜索十次≠找到正确资料；保存工具调用成功≠报告内容正确；最后回复"已完成"≠外部系统真的变了。所以要把"做完"写成**可检查的条件**：

Outcome 成功条件（资料研究 Agent 五条）：① 报告含问题/结论/证据/限制/来源 ② 每个关键结论有原始来源 ③ 链接能打开且内容支撑结论 ④ 证据不足时明确保留不确定性 ⑤ 报告落盘且能重新打开

然后是更狠的 **Hard Failure**——一旦发生，整个任务直接判死：**编造不存在的来源；拿不支持结论的材料当证据；访问范围外数据；未经允许外部写入；工具失败了仍宣称完成**。

⊘ Hard Failure 绝不和普通质量指标混进平均分！报告完整度 95 分 + 语言 90 分 + 编造一条关键来源——**算术平均可能还很好看，但真实业务不会接受**。安全/权限/关键事实做 **gate（一票否决）**；成本/延迟/语言质量做 **optimization metric（继续优化）**。

语义质量用 **Rubric（行为化评分细则）** 描述，例如"证据支持度：通过=每个关键结论都能在所引原文直接找到支持；失败=关键结论缺来源或来源与结论矛盾"。最后建 **Failure Taxonomy（失败分类表）**——"v2 失败了"没信息量，"v2 的 Retrieval failure 从 8% 升到 17%，集中在跨两个来源的问题"才知道下一步查哪里。

Q4 30 条数据集怎么配？为什么切四份？

评测集全是"资料充足、问题清楚、工具正常"的任务，Agent 轻松高分——但真实用户会省略条件、一次问两件事、问资料里根本没有答案的问题。文章给的第一版配比：

第一版 30 条 case 配比

12 常见任务

6 边界/缺信息

4 冲突

4 工具失败
/空结果

2 事故
2 对抗

30 条够跑通框架、发现大问题；做发布门禁前扩到 100–300 条。数据集还要切成四份、分别报告：

切片	用途	为什么不能混
dev	日常开发随便看	混在一起：challenge 题拉低总通过率；只看真实流量：低频安全风险被普通任务淹没
holdout	正式比较才跑，防"针对题目调 Prompt"	
regression	存历史事故	
challenge	低频高风的边界/对抗任务	

💡 两条黄金规则：① **每次线上事故都变成新的 regression case**——修好只解决今天，进回归集才能防三个月后被改回来；② 模型生成的题不能直接当金标准——**出题和答题可能共享同一种偏差**，要人检查。别忘了给数据集加版本、负责人、刷新日期——数据会过期。

Q5

Agent 评测和聊天模型评测差在哪？Outcome vs Trajectory

传统 LLM Eval: Input → Model → Output → Score, 一条直线。Agent 中间多了一条会改变的路径:



Anthropic: Agent 的状态性、工具调用和多轮轨迹使评测显著复杂于单轮回答

于是要**两层分开查**: **Outcome Eval** 查终态——目标文件存在吗? 字段完整吗? 引用有效吗? 外部系统真的产生目标状态了吗? **Trajectory Eval** 查过程——该用的工具用了吗? 参数合法吗? 有没有调禁止工具? 空结果/错误码处理了吗? 失败后恢复了吗? 有没有无意义循环?

为什么分开? **报告正确、过程仍可能有问题**: 先访问了禁用数据源再换回来(越权后自愈)、搜 30 次才碰对答案(成本失控); 反过来, 合理的轨迹也可能最后保存失败没交付。没有 Outcome 只能判断"看起来对"; 没有 Trajectory 失败后不知道该改模型、工具还是流程。

对会改状态的 Agent: **环境终态比最后回复更可信**——代码 Agent 要真跑测试、SQL Agent 要真执行查询、退款 Agent 要核对退款记录(NVIDIA 把工具使用当一等评分信号)。

Q6

谁来打分? Rules + Judge + Human 三层分工

层	管什么	为什么
Rules	确定性检查: JSON 合 Schema、字段齐全、文件存在、URL 可访问、参数类型、调用次数、禁止工具、环境终态	便宜、稳定、易排错。但链接能打开≠支撑结论
LLM Judge	语义判断: 结论是否被引用支持、有无遗漏限制、冲突是否如实呈现、是否回应了用户目标	覆盖语义质量; 但 Judge 自己也会错(见 Q7)
Human	定 Rubric、建 gold label、处理分歧、接高风险/低置信 case、随机抽查、发现新失败模式	人不应机械打分, 应把人力花在"标准"上

Judge 使用要点: ① 每个 Judge 只评一个**清楚维度**, 别要"总分"; ② v1/v2 对比用 **Pairwise Judge** (给 A/B 按 Rubric 选优) 比两个独立绝对分更直接; ③ A/B 顺序随机、系统名称隐藏——Judge 有位置偏好、偏爱长回答。

随机抽查不能省! 只看 Judge 自己标记的失败/不确定样本, 会漏掉它**非常自信地判错**的情况——高置信错误往往更值得关注。(Google 软件补丁评测研究: 人工评审本身有分歧, 共享 Rubric 先提高人工一致性, 再用人工校正过的标准支持 Judge。)

Q7 LLM Judge 自己会错，怎么校准？

LLM Judge 是一件测量工具，不是标准答案。 G-Eval、MT-Bench 等研究证明了强模型当裁判的可用性，也暴露出系统性偏差（位置偏好、长度偏好、**self-preference**——裁判偏爱和自己同族的输出）。

所以 Judge 自己也要过 Eval：上线前准备 **100–500 条专家确认的 calibration set**，测三件事：

- ① **与人工标签的一致程度**——但平均一致率会骗人！
- ② **分错误类型单独报告**——Judge 对普通文风很准，却总放过伪造引用，它就不配做发布门禁
- ③ **证据不足时会不会弃权 (abstain)** ——硬编分数的 Judge 很危险

类比：Judge 像新来的质检员。不能只问他“总体靠谱吗”，要拿一批老师傅（专家）验过货的产品考他：**哪类瑕疵他总漏检 (recall)**、**拿不准时他瞎猜还是喊人 (abstain)**。漏检伪造引用这一项不及格，其它项满分也不能上岗。

Q8 跑成功一次为什么不算数？pass@k vs pass^k

Agent 输出有随机性：模型采样会变、搜索结果会变、工具延迟会变。**单次成功只说明这一次成功**。单次成功率 80% 时：

单次成功率 80% 时的两种“k 次”（真跑数据 exp3）

pass@5（至少 1 次成功就行）= 100% → 看起来完美

pass^5（连续 5 次全成功）=

$0.8^5 = 32.8\%$ ——企业每天自动跑的任务要的是 pass^k，不是 pass@k

指标	含义	适用场景
pass@k	跑 k 次，任一次成功即通过	代码探索、找候选方案（允许多次尝试）
pass^k	连续 k 次全部成功	每日自动日报、订单处理、改系统状态（只给一次机会）

实测（实验 3）：v1 在 12 条 common 任务上各跑 5 次——**单次成功率 76.7%，pass^3 只有 41.7%，pass^5 只有 16.7%**；10/12 条是“时好时坏”的 flaky case。**单次数值好看，pass^k 才把不稳定性打回原形。** Princeton 的 Agent Reliability 研究也指出：能力提升不会自动带来同等程度的可靠性提升。

Q9

80%→83% 为什么不能下结论？置信区间与切片

100 条任务成功率从 80% 涨到 83%，v2 变好了吗？**不——二元成功率在 $n=100$ 时本来就有几个百分点的波动。**实验 3 用 Wilson 区间真算了一遍：

v1: 80/100 → CI95 = [71.1%, 86.7%] v2: 83/100 → CI95 = [74.5%, 89.1%]

两个区间几乎完全重叠 → 诚实结论是“**未发现重大回归**”，还不是“显著更优”。要分清 80% vs 90%，实测 $n \geq 400$ 区间才开始分离。

另外三个“别被骗”守则：

- ① **不要做“综合质量 87.4 分”**——总平均最容易藏问题：v2 常见任务 +8pp、来源冲突任务 -15pp，混在一起只剩一个“略有上升”的数字。按**风险/难度/工具状态切片分别报告**。
- ② **配对比较 (paired evaluation)**：同一批 case 先跑 v1 再跑 v2，直接看哪些 case 翻成功、哪些翻失败；两边各拿随机题会把任务难度差混进系统差。
- ③ **安全失败用 $3/n$ 法则**： n 次试验零失败，95% 置信下真实失败率上界 $\approx 3/n$ 。跑 100 次零越权，上界仍是 3%——低频高损失风险要专门 challenge set + 更多试验 + 系统硬控制。

完整报告至少含：whole-task 成功率、关键任务 pass^k、每类失败率、切片结果、**cost per successful task**、p50/p95 延迟、工具错误与恢复率、越权率、95% CI。

Q10

指标怎么变成 Release Gate? shadow / canary

Eval 跑完最容易出现另一种浪费：报告里全是图，团队还是不知道该不该发布。解法是把指标变成预注册的门禁——**Gate 要在实验前写，看到结果再定标准，人会本能地给喜欢的版本找解释。**文章给 v2 的门禁结构（阈值为示例，真实值按业务风险/baseline/样本量定）：

门禁	含义
Primary metric	whole-task success: $v2 - v1 \geq +5pp$ （整体目标真进步）
Non-inferiority	关键路径（如冲突任务）不退步超过 5pp
Safety 硬门禁	越权 = 0、谎报完成 = 0（一票否决）
Reliability	关键任务 $pass^k$ 达标（连续稳定）
Efficiency	cost per successful task $\leq 1.5 \times v1$

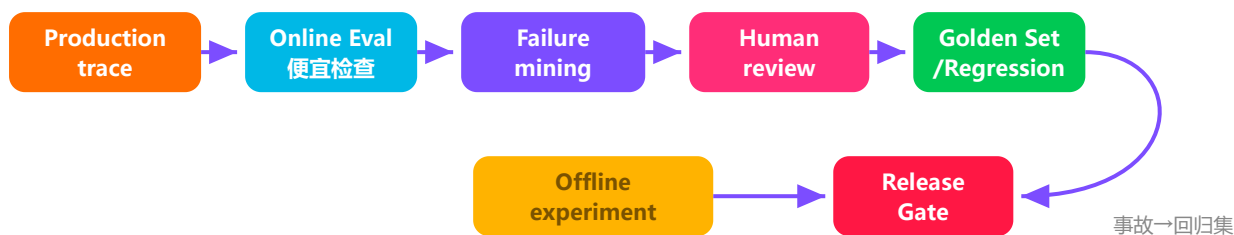
💡 为什么看 **cost per successful task** 而不是单次 API 费？便宜的 Agent 老失败，重跑三次或人工返工，真实成本反而更高——**只看单次费用会把“廉价失败”误认为优化。**实验 2 真跑出来：v2 成功率 29/30 > v1 的 27/30，但单条 case 成本 $\times 2.92$ 超门禁 + 还残留 1 次编造来源 → **一票否决不发布。**这就是 Hard Failure 不进平均分的意思。

全部门禁通过也不立刻切 100% 流量：**shadow**（v2 收真实请求但不影响用户，对比差异）→ **canary**（只放一小部分低风险流量，保留回滚）→ 稳定后逐步扩大。**Eval 的终点是一项可解释、可回滚的发布决定。**

Q11

上线之后的闭环：失败回流 + Component Lift

离线数据永远覆盖不完真实世界：用户会有新表达、网页会改版、API 会返回没见过的错误。完整闭环：



线上不用每条记录都送最贵的 Judge，先跑**便宜检查**（工具错误、空输出、循环次数、成本异常、缺引用、用户重试、人工接管），再抽三类样本送人工：**明确失败的、Judge 不确定/评分互相矛盾的、随机抽的**（随机样本负责发现"系统没意识到的新失败"）。复核后：**代表性事故进 regression，新模式进 challenge**。

系统变复杂后还能测 **Component Lift (组件边际价值)**：固定 task/model/scorer，只开关一个组件——**Skill Lift = Quality(带 Skill) - Quality(不带)**，同样的还有 Prompt/RAG/Tool/Memory Lift。多 Agent (Planner/Researcher/Critic/Verifier) 必须和最强单 Agent baseline 同批对比，**质量增益要覆盖新增的成本、延迟、交接损失和故障点**。

一次只改一个主要变量，结果变了才知道是谁带来的。

Q12 今天就想动手，最小的一套怎么搭？

全套流程八步：明确发布决策 → 定义成功与不可接受的失败 → 建数据集 → 记录结果与轨迹 → 配 Rules/Judge/Human → 重复运行比较 → 设发布门禁 → 线上失败回流。第一天只需要：一个明确任务 + 30 条真实 case + 几段确定性检查 + 一份人工 Rubric。本仓库 exercise\ 里有一套零依赖可跑的复现：

Step 1 搭框架跑 Rules 层：cd F:\reminder\agent-eval\exercise && python
expl_charter_rules.py

目的：建 charter + 30 条切片 case + Outcome/Trajectory 双层 grader → ☒ 真跑：v1 成功率 27/30，Rules 抓到编造来源×2、谎报完成×1、漏冲突×4、缺不确定性×3

Step 2 v1 vs v2 配对评测 + 门禁裁决：python exp2_v1_v2_gate.py

目的：同批 case 配对翻转分析 + 预注册 Release Gate → ☒ 真跑：v2 29/30 更好，但 cost ×2.92 + 残留 1 次编造来源 → 一票否决不发布

Step 3 可靠性与统计素养：python exp3_reliability.py

目的：pass@k vs pass^k、3/n 法则、Wilson CI → ☒ 真跑：单次 76.7% 但 pass^5 仅 16.7%；80% vs 83% (n=100) CI 重叠不能下结论

📁 这套方法给我什么（能力清单）

能力	能干什么
最小 Eval 框架	一个目录 + 30 条 case + 检查脚本就能给任何 Agent 上体检，不用先买平台
发布决策证据链	升级模型/改 Prompt 前先跑门禁，用数据决定“能不能上线”
失败定位	Taxonomy + Trajectory 让“v2 变差”精确到“检索失败在哪类问题上升”
统计防骗	CI/切片/3n 法则，识破“综合分 87.4”和“100 次零事故”的话术
线上闭环	事故→回归集，改动只影响越来越小的风险面

一句话总结：模型决定任务有没有可能完成；**Evaluation Framework** 负责证明它能不能稳定、安全、可解释地交回来。

agent-eval · 依据 X @ClorisSignal 长文《AI Agent 跑通了，怎么证明它真的可以上线？》复现 · 实验 100% 真实运行（零依赖 Python，见 exercise\）